SOAP Basics
3. SOAP Messages

SOAP message syntax is described in detail at http://www.w3.org/TR/SOAP/
Here is a quick summary and a few examples for the impatient.

A valid SOAP Message is a well-formed XML document. (For more detail on XML and well-formedness visit http://www.w3.org/XML/). The XML prolog can be present but, if present, should contain only an XML Declaration (ie. it should not contain any DTD references or XML processing instructions). It should use the SOAP Envelope and SOAP Encoding namespaces and have the following form:

- An XML Declaration (optional), followed by
- A SOAP Envelope (the root element) which is made up of:
  - A SOAP Header (optional)
  - A SOAP Body

A SOAP-encoded RPC dialogue contains both a request message and a response message. Let's consider a simple service method that doubles a given integer.

Method Signature

```
int doubleAnInteger ( int numberToDouble );
```

Request

```
<?xml version="1.0" encoding="UTF-8" standalone="no" ?>
<SOAP-ENV:Envelope
 SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
 xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
 xmlns:SOAP-ENC="http://schemas.xmlsoap.org/soap/encoding/"
 xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
 xmlns:xsd="http://www.w3.org/1999/XMLSchema">
      <SOAP-ENV:Body>
              <ns1:doubleAnInteger
               xmlns:ns1="urn:MySoapServices">
                      <param1 xsi:type="xsd:int">123</param1>
              </ns1:doubleAnInteger>
      </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Response

```
<?xml version="1.0" encoding="UTF-8" ?>
```

```
<SOAP-ENV:Envelope
 xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
 xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
 xmlns:xsd="http://www.w3.org/1999/XMLSchema">
      <SOAP-ENV:Body>
              <ns1:doubleAnIntegerResponse
               xmlns:ns1="urn:MySoapServices"
               SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
                     <return xsi:type="xsd:int">246</return>
              </ns1:doubleAnIntegerResponse>
      </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Note that most SOAP packages that you use will take care of the SOAP message syntax details for you, but for the sake of understanding and being able to read and debug SOAP dialogues, let's take a look at these messages piece by piece.

The XML prolog contains only an XML declaration **<?xml version="1.0" encoding="UTF-8" ?>**specifying the XML version and the character encoding of the XML message.

The SOAP Envelope tag **<SOAP-ENV:Envelope ... >**in the request message first specifies that this SOAP message's encoding style follows the schema defined at[http://schemas.xmlsoap.org/soap/encoding/](). Note that this is optional and will be presumed if not included as is the case in the response message. The SOAP Envelope tag also contains many namespace definitions. The namespace identifiers are standard and the SOAP specification requires that these namespaces either be defined correctly or not at all (ie. a SOAP message with missing namespace definitions is correct and processable but one with incorrect, ie. non-standard, definitions is incorrect and discardable). Notice that the SOAP-ENC namespace definition is missing from the response message but this does not mean the message is invalid. For more detail on XML namespaces visit [http://www.w3.org/TR/REC-xml-names/]().

There is no SOAP Header tag in the example. SOAP Headers are optional and are typically used to transmit authentication or session management data. It is important to remember that authentication and session management are out of the scope of the SOAP protocol although the designers of SOAP did allow for some flexibility in SOAP messaging so that implementors could include such information.

Then comes the SOAP Body tag **<SOAP-ENV:Body>** which is not remarkable in itself but encapsulates a single method tag porting the name of the method itself **<ns1:doubleAnInteger ... >**(or the same name suffixed with the word "Response" in the case of the response message). Note that the method tag is typically namespaced by the service name, in this case**urn:MySoapServices** to ensure uniqueness (A web service, which can contain any number of differently named methods, has a service name unique to the URL at which it is accessible - more detail on service URLs as well as service and method names can be found in the section on [Server-Side SOAP]()).

The method tag in turn encapsulates any number of parameter tags such as the **<param1 ... >** tag in the request envelope. Parameter tag names can be anything at all and are typically autogenerated and have no namespace. In the response message there is only ever

one parameter tag (representing the return value of the method) and it is typically named **<return>**.

More complex examples...

One of the most powerful features of the SOAP protocol is its ability to handle parameters of any level of complexity. This basically boils down to the ability to handle primitive types (int, string, etc.), arrays and structs and any combination thereof. Let's take a look at what the SOAP dialogues look like for methods with parameters and return types that are complex.

Below is the dialogue resulting from a call to the initial version of *getEmployeeDetails* as described in the introductory section What is SOAP? In this version the client sends an int and receives an array of strings (a two-element array of strings holding the employee name and the telephone number). Notice the type information contained in the **<return>** tag of the response message, namely **xsi:type="ns2:Array" ns2:arrayType="xsd:string[2]"**, which describes the structure of the array.

Method Signature

```
String[] getEmployeeDetails ( int employeeNumber );
```

Request

```
<?xml version="1.0" encoding="UTF-8" standalone="no" ?>
<SOAP-ENV:Envelope
 SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
 xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
 xmlns:SOAP-ENC="http://schemas.xmlsoap.org/soap/encoding/"
 xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
 xmlns:xsd="http://www.w3.org/1999/XMLSchema">
      <SOAP-ENV:Body>
              <ns1:getEmployeeDetails
               xmlns:ns1="urn:MySoapServices">
                      <param1 xsi:type="xsd:int">1016577</param1>
              </ns1:getEmployeeDetails>
      </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Response

```
<?xml version="1.0" encoding="UTF-8" ?>
<SOAP-ENV:Envelope
 xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
 xmlns:xsd="http://www.w3.org/1999/XMLSchema"
 xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
      <SOAP-ENV:Body>
              <ns1:getEmployeeDetailsResponse
```

```
                xmlns:ns1="urn:MySoapServices"
                SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
                        <return
                        xmlns:ns2="http://schemas.xmlsoap.org/soap/encoding/"
                        xsi:type="ns2:Array"
                        ns2:arrayType="xsd:string[2]">
                                <item xsi:type="xsd:string">Bill Posters</item>
                                <item xsi:type="xsd:string">+1-212-7370194</item>
                        </return>
                </ns1:getEmployeeDetailsResponse>
        </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Later on we made the service a little more complex. We wanted to provide a more complete set of contact numbers over time as employees came and went on business trips. The dialogue resulting from a call to this revised version of *getEmployeeDetails* is below. The client still sends an int but receives a complex type. Notice the nesting of the data in the response message and its correlation with the type definitions.

Method Signature

```
EmployeeContactDetail getEmployeeDetails ( int employeeNumber );
```

where the complex type *EmployeeContactDetail* is defined as

```
EmployeeContactDetail {
     String employeeName;
     String phoneNumber;
     TemporaryPhoneNumber[] tempPhoneNumber;
}
```

and the complex sub-type *TemporaryPhoneNumber* is defined as

```
TemporaryPhoneNumber {
     int startDate; //julian date
     int endDate;   //julian date
     String phoneNumber;
}
```

Request

```
<?xml version="1.0" encoding="UTF-8" standalone="no" ?>
<SOAP-ENV:Envelope
 SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
 xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
```

```
 xmlns:SOAP-ENC="http://schemas.xmlsoap.org/soap/encoding/"
 xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
 xmlns:xsd="http://www.w3.org/1999/XMLSchema">
      <SOAP-ENV:Body>
             <ns1:getEmployeeDetails
              xmlns:ns1="urn:MySoapServices">
                    <param1 xsi:type="xsd:int">1016577</param1>
             </ns1:getEmployeeDetails>
      </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Response

```
<?xml version="1.0" encoding="UTF-8" ?>
<SOAP-ENV:Envelope
 xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
 xmlns:xsd="http://www.w3.org/1999/XMLSchema"
 xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Body>
      <ns1:getEmployeeDetailsResponse
       xmlns:ns1="urn:MySoapServices"
       SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
          <return xsi:type="ns1:EmployeeContactDetail">
              <employeeName xsi:type="xsd:string">Bill Posters</employeeName>
              <phoneNumber xsi:type="xsd:string">+1-212-7370194</phoneNumber>
              <tempPhoneNumber
               xmlns:ns2="http://schemas.xmlsoap.org/soap/encoding/"
               xsi:type="ns2:Array"
               ns2:arrayType="ns1:TemporaryPhoneNumber[3]">
                  <item xsi:type="ns1:TemporaryPhoneNumber">
                      <startDate xsi:type="xsd:int">37060</startDate>
                      <endDate xsi:type="xsd:int">37064</endDate>
                      <phoneNumber xsi:type="xsd:string">+1-515-2887505</phoneNumber>
                  </item>
                  <item xsi:type="ns1:TemporaryPhoneNumber">
                      <startDate xsi:type="xsd:int">37074</startDate>
                      <endDate xsi:type="xsd:int">37078</endDate>
                      <phoneNumber xsi:type="xsd:string">+1-516-2890033</phoneNumber>
                  </item>
                  <item xsi:type="ns1:TemporaryPhoneNumber">
                      <startDate xsi:type="xsd:int">37088</startDate>
                      <endDate xsi:type="xsd:int">37092</endDate>
                      <phoneNumber xsi:type="xsd:string">+1-212-7376609</phoneNumber>
                  </item>
              </tempPhoneNumber>
```

```
            </return>
        </ns1:getEmployeeDetailsResponse>
    </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

In the preceding sections of SOAP Basics we learnt what SOAP is and how it can be used as the message protocol for the Service Web. In this section we looked at what the messages actually look like, and indeed what the *dialogues* actually look like, when a SOAP-based web service is consumed. In the final section on SOAP Basics we will take a look at what the different system elements are that go to make up a SOAP-based system, what each of them does and how they interact.

[Nicholas Quaine]